

1. INTRODUCTION

Chronic diseases such as cardiovascular disease and diabetes are among the major causes of long-term health complications and premature mortality. Early identification of individuals who are at risk can support timely medical intervention and help reduce the likelihood of severe complications. However, predicting disease risk from patient information can be challenging because healthcare data often contains multiple factors, including clinical measurements, lifestyle-related attributes, demographic information, and laboratory results.

Machine Learning provides an effective approach for analysing such data and identifying patterns that may not be easily recognised through manual analysis. Traditional disease prediction systems often rely on a single learning algorithm. However, different machine learning paradigms have different strengths and limitations. A neural network can learn complex non-linear relationships, a Genetic Algorithm can explore and optimise candidate solutions, and Bayesian reasoning can provide probabilistic predictions based on observed data.

This project proposes an **Adaptive Multi-Paradigm Learning Framework for Early Chronic-Disease Risk Prediction**, which combines these complementary approaches into a unified prediction framework. The system consists of a Multilayer Perceptron (MLP) implemented from first principles using NumPy, a Genetic Algorithm (GA) designed to optimise the MLP's initial weight configuration and/or hyperparameters, and a Naive Bayes classifier implemented independently as a probabilistic baseline.

The Multilayer Perceptron learns complex relationships between patient features and disease risk through forward propagation and back-propagation. Instead of relying only on randomly initialised model parameters, a Genetic Algorithm is used to search for improved initial configurations or hyperparameter values. This allows the study to investigate whether evolutionary optimisation can improve the convergence and predictive performance of the neural network.

In addition, a Naive Bayes classifier is developed from first principles to provide an independent probabilistic approach to disease prediction. The predictions produced by the optimised MLP and Naive Bayes models are then combined using a mathematically justified decision-fusion mechanism. The objective of this approach is to investigate whether combining complementary learning strategies can provide improved and more robust predictions compared with relying on a single model.

The proposed framework is evaluated using a publicly available clinical dataset containing patient health-related attributes and disease labels. The dataset is subjected to preprocessing to address missing or noisy values, feature scaling, and class imbalance before model development. The different configurations, including the baseline MLP, Genetic Algorithm-optimised MLP, and fused prediction

model, are evaluated using k-fold cross-validation and performance measures such as accuracy, precision, recall, F1-score, and ROC-AUC.

Beyond predictive performance, the project also considers the computational learning aspects of the proposed framework by analysing sample complexity and discussing the relationship between the available dataset size and PAC-learning concepts. Ethical considerations, including patient privacy, fairness, bias, transparency, and responsible deployment, are also examined in relation to the use of Artificial Intelligence in healthcare.

1.1 Background and Motivation

The increasing availability of digital healthcare records has created opportunities to apply Machine Learning techniques for disease prediction and risk assessment. Patient datasets can contain valuable information related to age, clinical measurements, lifestyle factors, laboratory test results, and other health indicators. Analysing these variables manually can be difficult when multiple factors interact in complex and non-linear ways.

Early prediction of chronic diseases is particularly important because many chronic conditions develop gradually and may initially show limited or non-specific symptoms. Identifying high-risk individuals at an earlier stage can support preventive measures, regular monitoring, and timely medical consultation.

Most basic Machine Learning solutions approach this problem using a single classification algorithm. While such models may provide acceptable performance, they may not fully exploit the advantages offered by different learning paradigms. Neural networks are capable of learning complex non-linear relationships, but their performance can be influenced by weight initialisation and hyperparameter selection. Genetic Algorithms provide a population-based optimisation strategy that can search for better candidate configurations. Bayesian classifiers, on the other hand, provide probabilistic predictions based on statistical relationships between features and target classes.

The motivation behind this project is therefore to investigate whether these complementary approaches can be combined into a single adaptive learning framework. Instead of depending on one model, the proposed system evaluates a baseline neural network, improves it using evolutionary optimisation, and combines its predictions with an independently developed probabilistic classifier.

This approach provides both a practical Machine Learning application and an opportunity to study the strengths and limitations of neural networks, Genetic Algorithms, Bayesian reasoning, and model fusion within a real-world healthcare prediction problem.

1.2 Problem Statement

A healthcare analytics system requires an early-warning mechanism capable of predicting the risk of a chronic disease from patient clinical, demographic, lifestyle, and laboratory-related information. Reliable prediction is important because early identification of high-risk individuals can support timely medical intervention and help reduce preventable complications.

Traditional Machine Learning approaches often rely on a single prediction algorithm, which may not fully capture the advantages offered by different learning strategies. Neural networks can model complex non-linear relationships but may be sensitive to initial weight configurations and hyperparameter choices. Genetic Algorithms can perform population-based optimisation to search for improved candidate solutions, while Bayesian classifiers provide an independent probabilistic approach to classification.

Therefore, the problem addressed in this project is the design and implementation of an adaptive multi-paradigm learning framework that combines:

1. A Multilayer Perceptron implemented from first principles using NumPy.
2. Back-propagation for learning neural network parameters.
3. A Genetic Algorithm implemented from scratch to optimise the MLP configuration.
4. A Naive Bayes classifier implemented independently from first principles.
5. A decision-fusion mechanism that combines the predictions of the optimised MLP and Naive Bayes models.

The system must be evaluated rigorously using k-fold cross-validation and appropriate performance metrics, including accuracy, precision, recall, F1-score, and ROC-AUC. The study must also analyse whether the improvements obtained through Genetic Algorithm optimisation and model fusion are statistically meaningful.

The final objective is to develop and evaluate a reliable Machine Learning framework for early chronic-disease risk prediction using complementary learning paradigms rather than relying on a single pre-built Machine Learning algorithm.

1.3 Research Gap

Many existing disease prediction approaches focus on applying a single Machine Learning model to a healthcare dataset and evaluating its predictive accuracy. Although this approach can produce useful results, it does not explore how different learning paradigms can complement one another.

Neural networks are capable of modelling complex non-linear relationships but may be affected by random weight initialisation and the selection of hyperparameters. Similarly, probabilistic classifiers such as Naive Bayes provide efficient and interpretable probability-based predictions but rely on simplifying assumptions regarding feature independence.

A gap therefore exists in the systematic integration and comparison of these approaches within a single framework. Specifically, there is value in investigating:

- Whether Genetic Algorithm-based optimisation can improve the convergence or predictive performance of a Multilayer Perceptron compared with random initialisation.
- How an independently implemented Bayesian classifier compares with a neural-network-based approach.
- Whether combining predictions from an optimised neural network and a probabilistic classifier through a mathematically justified fusion mechanism can improve prediction performance.
- Whether the observed improvements remain consistent across multiple validation folds and are statistically significant.
- How computational learning theory concepts such as sample complexity and PAC-learning can be related to the practical dataset and hypothesis space used in the project.

The proposed framework addresses this gap by combining neural learning, evolutionary optimisation, and Bayesian reasoning and evaluating the individual and fused approaches under a common experimental setting.

1.4 Aim

The aim of this project is to design, implement, and evaluate an adaptive multi-paradigm Machine Learning framework for early chronic-disease risk prediction by integrating a Multilayer Perceptron, Genetic Algorithm-based optimisation, Naive Bayes classification, and decision fusion using a publicly available clinical dataset.

1.5 Objectives

The objectives of the proposed study are:

1. To collect and analyse a publicly available clinical dataset containing at least 500 patient records for chronic-disease risk prediction.
2. To preprocess the dataset by handling missing or noisy values, preparing features, applying appropriate scaling, and addressing class imbalance.

3. To implement a Multilayer Perceptron from first principles using NumPy, including forward propagation, at least one hidden layer, and back-propagation.
4. To establish a baseline neural network using conventional random weight initialisation.
5. To design and implement a Genetic Algorithm from scratch with appropriate fitness evaluation, selection, crossover, and mutation operators.
6. To use the Genetic Algorithm to optimise the MLP's initial weight configuration and/or selected hyperparameters.
7. To empirically compare the convergence and predictive performance of the baseline MLP and the Genetic Algorithm-optimised MLP.
8. To implement a Naive Bayes classifier from first principles as an independent probabilistic baseline.
9. To manually derive and demonstrate the posterior probability calculation for at least one test patient record.
10. To design a mathematically justified decision-fusion mechanism combining the predictions of the MLP+GA model and the Naive Bayes classifier.
11. To evaluate the baseline MLP, MLP+GA, Naive Bayes, and fused model using k-fold cross-validation.
12. To measure and compare performance using accuracy, precision, recall, F1-score, and ROC-AUC.
13. To perform statistical analysis to determine whether the observed performance improvements are statistically significant.
14. To analyse the computational learning aspects of the proposed framework using sample-complexity estimation and PAC-learning concepts.
15. To examine ethical, privacy, fairness, and responsible deployment considerations and relate the proposed work to Sustainable Development Goal 3: Good Health and Well-Being.

2. LITERATURE REVIEW

Machine Learning has increasingly been applied in healthcare to analyse patient data and support disease prediction. Different approaches have been proposed using neural networks, probabilistic classifiers, optimisation algorithms, and hybrid learning frameworks. The following section reviews the concepts and approaches most relevant to the proposed Adaptive Multi-Paradigm Learning Framework.

2.1 Machine Learning for Chronic Disease Prediction

Chronic disease prediction is generally formulated as a supervised classification problem in which patient-related features are used to predict the presence or risk of a particular disease. Healthcare datasets may contain demographic information, clinical measurements, laboratory values, lifestyle-related variables, and medical history.

Machine Learning algorithms can analyse relationships between these variables and identify patterns associated with disease outcomes. However, healthcare datasets also introduce challenges such as missing values, noisy measurements, correlated features, class imbalance, and limited sample sizes. Therefore, the performance of a disease prediction system depends not only on the selected learning algorithm but also on appropriate data preprocessing and validation.

2.2 Multilayer Perceptron for Disease Prediction

Artificial Neural Networks are widely used for classification tasks involving complex and non-linear relationships. A Multilayer Perceptron consists of an input layer, one or more hidden layers, and an output layer.

During forward propagation, the input features are transformed through weighted connections and activation functions to generate a prediction. The difference between the predicted output and the actual target value is measured using a loss function. Back-propagation then calculates the gradients of the loss with respect to the network parameters and updates the weights to reduce prediction error.

For healthcare prediction problems, an MLP can learn non-linear relationships between multiple patient attributes and disease risk. However, neural network performance can be influenced by factors such as initial weight configuration, learning rate, network architecture, and training procedure.

The proposed study therefore establishes a baseline MLP using conventional random initialisation and compares it with a Genetic Algorithm-optimised version.

2.3 Genetic Algorithms for Neural Network Optimisation

A Genetic Algorithm is an evolutionary optimisation technique inspired by the process of natural selection. Instead of improving a single solution directly, a GA maintains a population of candidate solutions.

Each candidate is evaluated using a fitness function. Higher-performing candidates are selected for reproduction, and new candidates are generated using crossover and mutation operations. Over successive generations, the population is expected to evolve towards better solutions.

In the context of neural networks, a Genetic Algorithm can be used to optimise initial weight configurations or selected hyperparameters. Random initialisation may lead the learning process to begin from less favourable regions of the optimisation space. A GA can explore multiple candidate configurations and identify potentially better starting points.

However, GA-based optimisation also introduces additional computational cost because multiple candidate solutions must be evaluated. Therefore, the proposed framework evaluates whether the improvement in convergence or predictive performance justifies this additional optimisation effort.

2.4 Naive Bayes for Probabilistic Disease Prediction

Naive Bayes is a probabilistic classification approach based on Bayes' theorem. It estimates the probability of a target class based on the observed input features.

For a feature vector X and class C , Bayes' theorem can be expressed as:

$$P(C|X) = [P(X|C) \times P(C)] / P(X)$$

The Naive Bayes classifier makes the simplifying assumption that individual features are conditionally independent given the class. Although this assumption may not always be fully valid for healthcare data, the method remains computationally efficient and provides a useful probabilistic baseline.

In the proposed system, Naive Bayes is implemented independently from first principles. This allows its probability calculations to be examined directly rather than treating the classifier as a black-box model.

The project also includes a manual posterior probability derivation for at least one test patient to demonstrate the underlying Bayesian reasoning process.

2.5 Hybrid and Decision-Fusion Approaches

Individual Machine Learning models have different strengths and limitations. A neural network may capture complex non-linear relationships, whereas a probabilistic classifier provides a statistically interpretable estimate of class likelihood.

Decision fusion combines predictions from multiple models to generate a final prediction. A common approach is probability-based fusion, in which the predicted probabilities from different models are combined using weighted averaging.

For example:

$$P(\text{final}) = w_1 P(\text{MLP+GA}) + w_2 P(\text{Naive Bayes})$$

where:

- **P(MLP+GA)** represents the predicted probability produced by the optimised neural network.
- **P(Naive Bayes)** represents the probability generated by the Bayesian classifier.
- **w₁ and w₂** represent the weights assigned to each model, where **w₁ + w₂ = 1**.

The weights can be determined using validation performance so that models demonstrating stronger predictive ability receive greater influence on the final decision.

The proposed framework uses decision fusion to investigate whether combining neural and probabilistic learning can provide a more robust prediction than either individual approach.

2.6 Identified Research Gap

From the reviewed approaches, it can be observed that neural networks, Genetic Algorithms, and Bayesian classifiers provide different perspectives for solving a classification problem.

However, a complete framework that:

- Implements an MLP from first principles rather than relying on a pre-built neural network library,
- Uses a Genetic Algorithm developed from scratch to optimise the neural network,
- Implements Naive Bayes independently from fundamental probability calculations,
- Compares baseline MLP and GA-optimised MLP convergence,
- Combines neural and Bayesian predictions through a justified decision-fusion mechanism,
- Evaluates the models using cross-validation and statistical analysis, and
- Relates practical results to computational learning theory concepts,

provides a more comprehensive experimental study of multiple Machine Learning paradigms.

Therefore, the proposed research addresses this gap through the development of an **Adaptive Multi-Paradigm Learning Framework for Early Chronic-Disease Risk Prediction**.

2.7 LITERATURE COMPARISON

Approach	Main Strength	Main Limitation	Role in Proposed Framework
Multilayer Perceptron	Learns complex non-linear relationships	Sensitive to initialisation and hyperparameters	Core predictive model
Genetic Algorithm	Explores a large solution space and performs global search	Computationally expensive	Optimises MLP configuration
Naive Bayes	Fast and probabilistic	Assumes independence between features	Conditional Independent probabilistic baseline
Single Model Prediction	Simple to implement and evaluate	May not utilise complementary strengths	Used as baseline comparison
Decision Fusion	Combines complementary predictions	Requires suitable fusion strategy	Final integrated prediction

The comparison shows the motivation for combining the selected approaches. The MLP provides non-linear learning capability, the Genetic Algorithm improves the search for suitable model configurations, and Naive Bayes provides an independent probability-based prediction. Decision fusion is then used to combine the complementary information produced by the two learning approaches.

3. DATASET COLLECTION AND UNDERSTANDING

3.1 Dataset Selection

For the proposed study, the **UCI Heart Disease Dataset** was selected as the primary dataset for developing the chronic-disease risk prediction framework.

The assignment requires a real, publicly available clinical dataset containing at least 500 records and specifically identifies the UCI Heart Disease Dataset as one of the suitable examples.

The UCI Heart Disease repository contains data collected from four clinical databases:

- Cleveland Clinic Foundation
- Hungarian Institute of Cardiology
- University Hospital, Zurich, Switzerland
- VA Medical Center, Long Beach

The repository provides separate data files for these sources. For this project, the compatible processed records from the available databases will be combined to construct a dataset containing approximately **920 patient records**, thereby satisfying the assignment requirement of at least 500 records. The UCI repository documents the four source databases and the relevant heart-disease attributes used for prediction.

The dataset is suitable for this study because it represents a real clinical classification problem involving demographic information, physiological measurements, electrocardiographic information, exercise-related observations, and diagnostic outcomes.

3.2 Dataset Source

The dataset was obtained from the **UCI Machine Learning Repository**.

Dataset:	Heart	Disease
Domain:	Health	and Medicine
Learning	Task:	Classification
Dataset	Type:	Multivariate
Feature Types: Categorical, Integer, and Continuous		

The original UCI dataset contains multiple attributes, but the commonly used prediction subset consists of 13 input attributes and one target variable.

Dataset Source: [UCI Machine Learning Repository – Heart Disease Dataset](#)

3.3 Dataset Description

The dataset contains patient records collected during clinical examinations related to heart disease. Each record contains information that can potentially contribute to the prediction of whether heart disease is present.

The commonly used attributes are shown below.

No. Attribute Description

1	age	Age of the patient in years
2	sex	Biological sex recorded in the dataset
3	cp	Chest pain type
4	trestbps	Resting blood pressure
5	chol	Serum cholesterol level
6	fbs	Fasting blood sugar greater than 120 mg/dl
7	restecg	Resting electrocardiographic result
8	thalach	Maximum heart rate achieved
9	exang	Exercise-induced angina
10	oldpeak	ST depression induced by exercise relative to rest
11	slope	Slope of the peak exercise ST segment
12	ca	Number of major vessels coloured by fluoroscopy
13	thal	Thalassemia-related test result
14	num	Diagnosis of heart disease

The num attribute represents the target variable. In the original dataset, values represent different levels of heart disease. For the proposed binary classification task, the target will be transformed as follows:

$$y = \begin{cases} 0, & \text{if num} = 0 \text{ (absence of heart disease)} \\ 1, & \text{if num} > 0 \text{ (presence of heart disease)} \end{cases}$$

This transformation converts the problem into a binary chronic-disease risk classification task suitable for the MLP, Genetic Algorithm optimisation, Naive Bayes classifier, and decision-fusion framework.

The UCI documentation describes num as the diagnostic target and documents the feature meanings and relevant subset of attributes.

3.4 Input Features and Target Variable

The dataset can be represented as:

Input Features

$$X = \{a, g, e, s, e, x, c, p, t, r, e, s, tbpscholfbsrestecgthalachexangoldpeakslopecathal\}$$

Target Variable

$$Y = num$$

After binary transformation:

$$Y \in \{0, 1\}$$

Where:

- **0** represents the absence of heart disease.
- **1** represents the presence of heart disease.

The resulting Machine Learning task is therefore a **binary supervised classification problem**.

3.5 Justification for Dataset Selection

The UCI Heart Disease Dataset was selected for the following reasons:

1. It represents a real healthcare and chronic-disease prediction problem.
2. The data is publicly available and suitable for academic experimentation.
3. The repository contains multiple clinical data sources that can be combined to satisfy the minimum dataset-size requirement specified in the assignment.
4. The dataset contains both numerical and categorical attributes, making preprocessing an important part of the Machine Learning pipeline.
5. The available records contain missing values in certain attributes, providing an opportunity to implement and evaluate data-cleaning strategies.
6. The dataset is suitable for binary classification after transforming the diagnostic target into presence or absence of heart disease.
7. The problem is appropriate for comparing neural learning, evolutionary optimisation, probabilistic classification, and decision fusion.

3.6 Initial Dataset Characteristics

Before model development, the combined dataset will be examined to determine:

- Total number of patient records.
- Number of input features.
- Data types of each attribute.
- Number and distribution of missing values.
- Duplicate records, if any.
- Statistical properties of numerical variables.
- Distribution of the target classes.
- Presence of possible class imbalance.
- Presence of noisy or extreme values.

This initial analysis is necessary because the assignment explicitly requires the Machine Learning pipeline to address missing or noisy data and class imbalance.

The exact descriptive statistics and class distribution will be generated from the actual combined dataset during the implementation stage and will be reported in the subsequent **Exploratory Data Analysis and Preprocessing** section.

3.7 Dataset Collection Procedure

The dataset collection procedure for the project is defined as follows:

Step 1: Obtain the four available heart-disease data sources from the UCI Machine Learning Repository.

Step 2: Identify the common attributes required for the classification task.

Step 3: Load each compatible dataset into a Pandas DataFrame.

Step 4: Assign consistent column names to all datasets.

Step 5: Combine the patient records into a single dataset.

Step 6: Convert missing-value indicators, such as ?, into a standard missing-value representation.

Step 7: Retain the selected 13 input attributes and the diagnostic target variable.

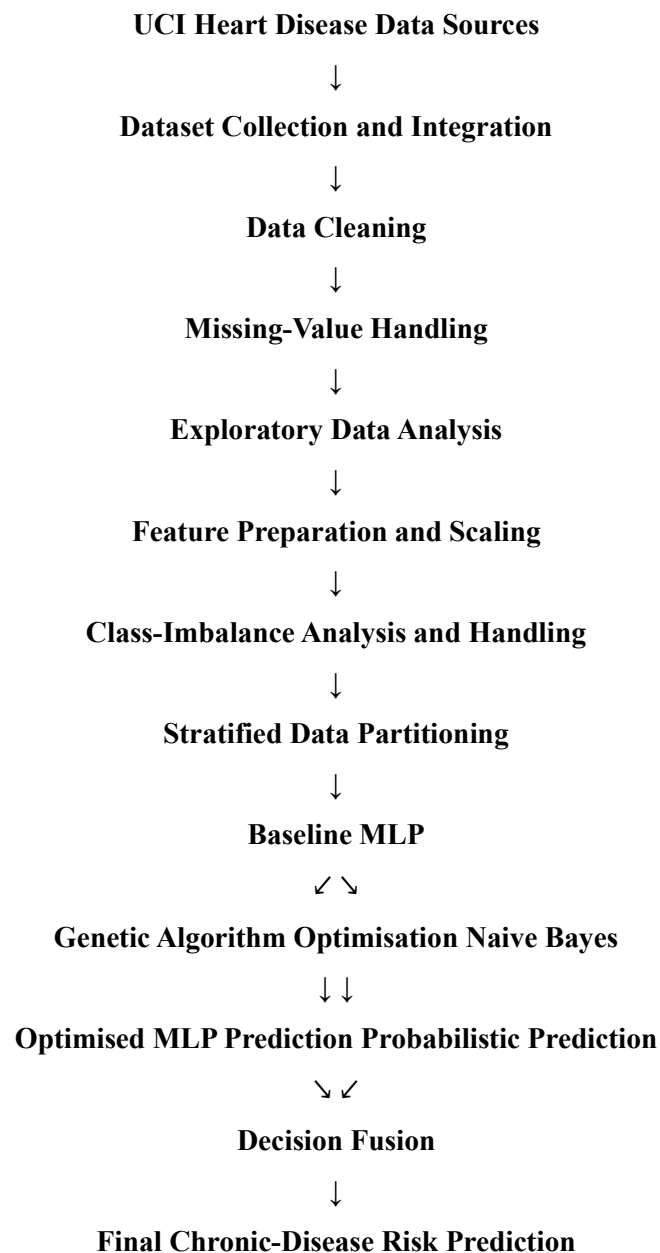
Step 8: Transform the original target variable into a binary output representing the absence or presence of heart disease.

Step 9: Perform exploratory analysis to understand the characteristics and quality of the combined dataset.

The final dataset produced through this process will be used consistently for all experimental configurations to ensure a fair comparison between the baseline MLP, GA-optimised MLP, Naive Bayes classifier, and fused prediction model.

3.8 Dataset and Research Pipeline

The dataset will enter the proposed Machine Learning framework through the following pipeline:



4. EXPLORATORY DATA ANALYSIS AND DATA PREPROCESSING

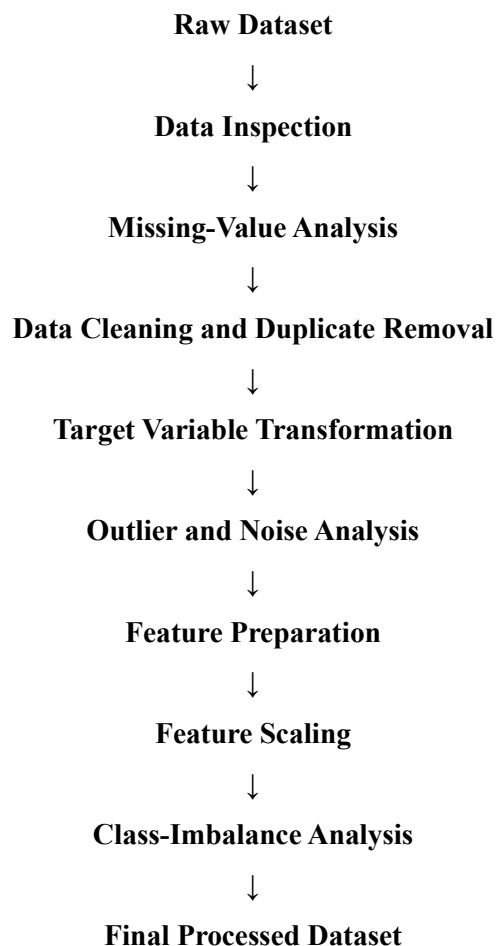
4.1 Overview

After collecting and integrating the selected heart-disease datasets, Exploratory Data Analysis (EDA) and data preprocessing are performed before developing the Machine Learning models.

Healthcare data may contain missing values, inconsistent representations, duplicate records, noisy observations, and features measured on significantly different numerical scales. If these issues are not addressed appropriately, they can negatively affect model training and evaluation.

Therefore, the preprocessing pipeline is designed to prepare a clean and consistent dataset for the proposed MLP, Genetic Algorithm optimisation, Naive Bayes classification, and decision-fusion framework.

The preprocessing workflow is:



4.2 Data Loading and Initial Inspection

The collected datasets are loaded using the Pandas library and integrated into a single structured dataset. The first stage of analysis involves examining:

- Dataset dimensions.
- Feature names.
- Data types.
- Sample records.
- Summary statistics.
- Missing values.
- Duplicate records.
- Target-class distribution.

The initial inspection helps identify potential data-quality issues before further processing.

The following operations are performed:

```
import pandas as pd
```

```
import numpy as np
```

```
# Display dataset shape
```

```
print("Dataset Shape:", df.shape)
```

```
# Display first five records
```

```
print(df.head())
```

```
# Display column information
```

```
print(df.info())
```

```
# Generate descriptive statistics
```

```
print(df.describe())
```

```
# Check missing values
```



```
print(df.isnull().sum())
```

```
# Check duplicate records
```

```
print("Duplicate Records:", df.duplicated().sum())
```

The results obtained from these operations will be recorded and used to describe the actual characteristics of the final combined dataset.

4.3 Handling Missing Values

Clinical datasets may contain missing values because certain medical tests or measurements may not have been recorded for every patient.

In the selected dataset, missing-value indicators such as ? are first converted into standard NaN values.

```
df.replace("?", np.nan, inplace=True)
```

The missing values are then identified using:

```
print(df.isnull().sum())
```

Numerical attributes containing missing values are handled using median imputation.

The median is selected because it is less sensitive to extreme values than the arithmetic mean.

```
numerical_columns = ["trestbps", "chol", "thalach", "oldpeak", "ca"]
```

```
for column in numerical_columns:
```

```
    df[column] = pd.to_numeric(df[column], errors="coerce")
```

```
    df[column] = df[column].fillna(df[column].median())
```

If missing values occur in categorical features, the most frequent value can be used for imputation.

```
categorical_columns = ["cp", "restecg", "slope", "thal"]
```

```
for column in categorical_columns:
```

```
    df[column] = df[column].fillna(df[column].mode()[0])
```

This approach allows the maximum number of available patient records to be retained while ensuring that missing values do not prevent model training.

4.4 Data Type Conversion

Some attributes may initially be loaded as strings because of inconsistent values or missing-value indicators.

Therefore, numerical attributes are explicitly converted into numerical data types.

```
columns = [  
    "age", "sex", "cp", "trestbps", "chol",  
    "fbs", "restecg", "thalach", "exang",  
    "oldpeak", "slope", "ca", "thal", "num"  
]
```

for column in columns:

```
    df[column] = pd.to_numeric(df[column], errors="coerce")
```

After conversion, the dataset is inspected again to ensure that all required variables are represented consistently.

4.5 Duplicate Record Analysis

Duplicate records can cause certain observations to have an unintended influence during model training.

The number of duplicate records is checked using:

```
duplicate_count = df.duplicated().sum()
```

```
print("Number of duplicate records:", duplicate_count)
```

If duplicate records are identified, they are removed:

```
df = df.drop_duplicates()
```

Removing duplicate records helps ensure that the training data contains unique observations and reduces unnecessary bias caused by repeated patient entries.

4.6 Target Variable Transformation

The original diagnostic target variable contains multiple values representing different levels of heart disease.

For this study, the problem is formulated as a binary classification task.

The transformation is defined as:

$$y = \begin{cases} 0, & \text{if } num = 0 \\ 1, & \text{if } num > 0 \end{cases}$$

This is implemented using:

```
df["target"] = (df["num"] > 0).astype(int)
```

```
df = df.drop(columns=["num"])
```

The resulting target variable represents:

Target Value Interpretation

0	Absence of heart disease
1	Presence of heart disease

This binary representation is used consistently across all proposed Machine Learning models.

4.7 Exploratory Analysis of Numerical Features

Descriptive statistics are generated to understand the distribution of numerical features.

The following properties are examined:

- Mean
- Median
- Minimum value
- Maximum value
- Standard deviation
- Distribution of values

```
print(df.describe())
```

Visualisations such as histograms and box plots can be generated to examine the distribution of clinical variables and identify potential extreme observations.

```
import matplotlib.pyplot as plt
```

```
df.hist(figsize=(12, 10))
```

```
plt.tight_layout()
```

```
plt.show()
```

The exploratory analysis helps identify whether specific variables contain unusual ranges or extreme values that may require further investigation.

4.8 Outlier and Noise Analysis

Healthcare datasets can contain unusually high or low measurements. However, an extreme value in clinical data is not automatically considered an error because it may represent a genuine patient condition.

Therefore, potential outliers are first identified rather than automatically removed.

The Interquartile Range (IQR) method is used for exploratory analysis.

$$IQR = Q_3 - Q_1$$

The lower and upper boundaries are calculated as:

$$\text{Lower Bound} = Q_1 - 1.5 \times IQR$$

$$\text{Upper Bound} = Q_3 + 1.5 \times IQR$$

The implementation is:

```
def detect_outliers(series):
```

```
    q1 = series.quantile(0.25)
```

```
    q3 = series.quantile(0.75)
```

```
    iqr = q3 - q1
```

```
    lower_bound = q1 - 1.5 * iqr
```

```
    upper_bound = q3 + 1.5 * iqr
```

```
    return series[
```

```
(series < lower_bound) |
(series > upper_bound)
]
```

for column in numerical_columns:

```
outliers = detect_outliers(df[column])

print(column, ":", len(outliers))
```

Potential outliers are examined carefully. Rather than removing all extreme clinical values, invalid or clearly inconsistent values are corrected only when justified by the data.

This approach prevents genuine high-risk patient records from being unnecessarily removed.

4.9 Feature Scaling

The selected dataset contains numerical variables measured on different scales. For example, age, cholesterol level, blood pressure, and maximum heart rate have substantially different ranges.

Feature scaling is particularly important for the MLP because gradient-based optimisation can be affected by differences in feature magnitude.

Standardisation is applied using:

$$x' = \frac{x - \mu}{\sigma}$$

where:

- x is the original feature value.
- μ is the mean of the feature.
- σ is the standard deviation.

For the final implementation, scaling statistics must be calculated using only the training portion of each fold to prevent data leakage.

The standardisation operation can be implemented manually as:

```
class StandardScalerScratch:
```

```
    def fit(self, X):
```

```

self.mean = np.mean(X, axis=0)

self.std = np.std(X, axis=0)

# Prevent division by zero

self.std[self.std == 0] = 1

def transform(self, X):

    return (X - self.mean) / self.std

def fit_transform(self, X):

    self.fit(X)

    return self.transform(X)

```

This ensures that the preprocessing procedure is implemented consistently and independently.

4.10 Class-Imbalance Analysis

Before model training, the distribution of the target classes is examined.

```

class_counts = df["target"].value_counts()

print(class_counts)

print(df["target"].value_counts(normalize=True) * 100)

```

The class distribution is visualised using:

```

df["target"].value_counts().plot(kind="bar")

plt.xlabel("Target Class")

plt.ylabel("Number of Patients")

plt.title("Distribution of Heart Disease Classes")

```

```
plt.show()
```

If a significant class imbalance is identified, appropriate strategies can be applied.

For this project, possible approaches include:

- Class-weighted loss during MLP training.
- Balanced sampling within training folds.
- Random oversampling of the minority class using a manually implemented sampling procedure.

Any balancing operation will be applied **only to the training data** within each validation fold. The test portion will remain unchanged to ensure realistic and unbiased evaluation.

4.11 Feature and Target Separation

After preprocessing, the input features and target variable are separated.

```
X = df.drop(columns=["target"]).values.astype(float)
```

```
y = df["target"].values.astype(int)
```

```
print("Feature Matrix Shape:", X.shape)
```

```
print("Target Vector Shape:", y.shape)
```

The feature matrix X contains the patient attributes, while the target vector y contains the corresponding binary disease labels.

Mathematically:

$$X \in \mathbb{R}^{n \times d}$$

where:

- n represents the number of patient records.
- d represents the number of input features.

The target is represented as:

$$y \in \{0, 1\}^n$$

4.12 Data Preparation for Experimental Evaluation

The final processed dataset will be evaluated using stratified k-fold cross-validation.

Stratification ensures that the approximate proportion of positive and negative disease cases is maintained across the validation folds.

The general procedure is:

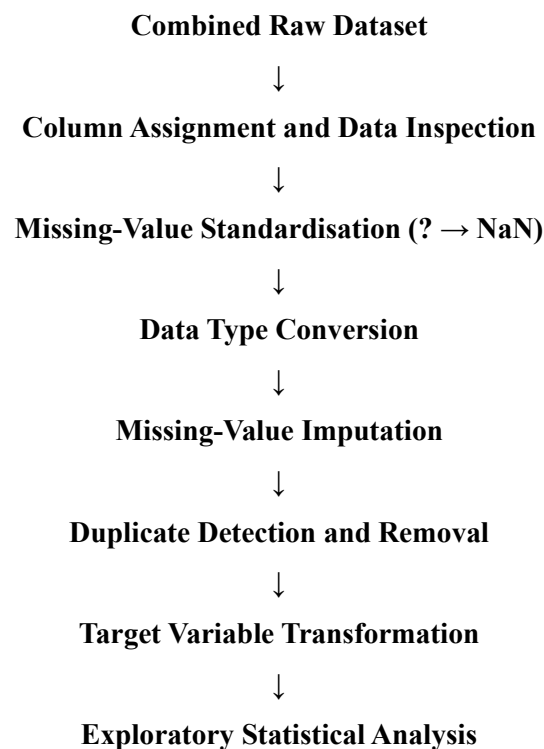
1. Divide the dataset into k folds.
2. Use $k - 1$ folds for training.
3. Use the remaining fold for testing.
4. Repeat the process until each fold has been used as the test set.
5. Calculate evaluation metrics for every fold.
6. Compute the mean and standard deviation of the results.

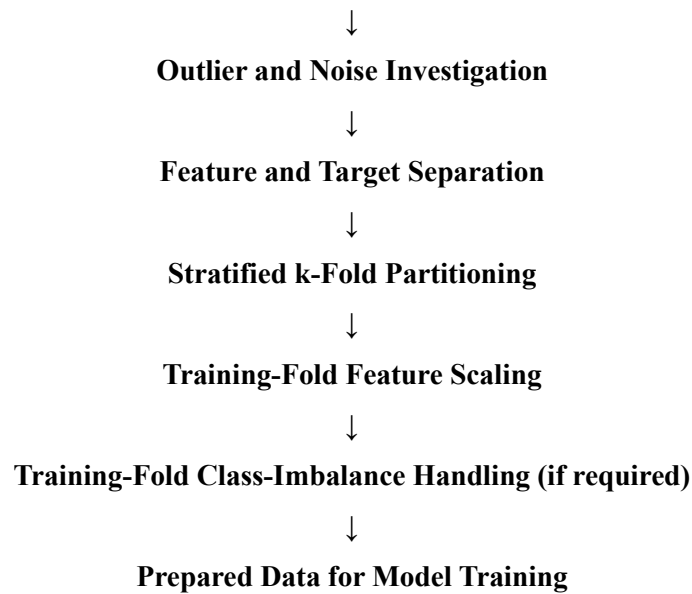
The preprocessing operations, including scaling and any class-balancing procedure, are fitted using only the training data in each fold and then applied to the corresponding test data.

This prevents information from the test data from influencing the training process.

4.13 Final Preprocessing Workflow

The complete preprocessing pipeline for the proposed system is shown below:





4.14 Summary

The data preprocessing stage prepares the raw clinical records for reliable Machine Learning experimentation. Missing values are standardised and handled, data types are converted into a consistent format, duplicate records are analysed, and the original diagnostic target is transformed into a binary classification variable.

Exploratory analysis is used to understand feature distributions, potential outliers, and class distribution. Feature scaling is applied carefully within each training fold to prevent data leakage, while class imbalance is analysed and addressed only when necessary.

The resulting processed dataset forms the common input for all experimental models: the baseline MLP, the Genetic Algorithm-optimised MLP, the Naive Bayes classifier, and the final decision-fusion model.

5. PROPOSED METHODOLOGY

5.1 Overview of the Proposed Framework

The proposed **Adaptive Multi-Paradigm Learning Framework for Early Chronic-Disease Risk Prediction** combines three different Machine Learning approaches: neural learning, evolutionary optimisation, and probabilistic reasoning.

The framework is designed to evaluate whether combining these approaches can improve chronic-disease risk prediction compared with using a single model.

The proposed methodology consists of four major stages:

1. **Baseline Multilayer Perceptron (MLP)**
2. **Genetic Algorithm (GA) Optimisation of the MLP**
3. **Naive Bayes Probabilistic Classification**
4. **Decision Fusion of MLP+GA and Naive Bayes Predictions**

The complete workflow is:

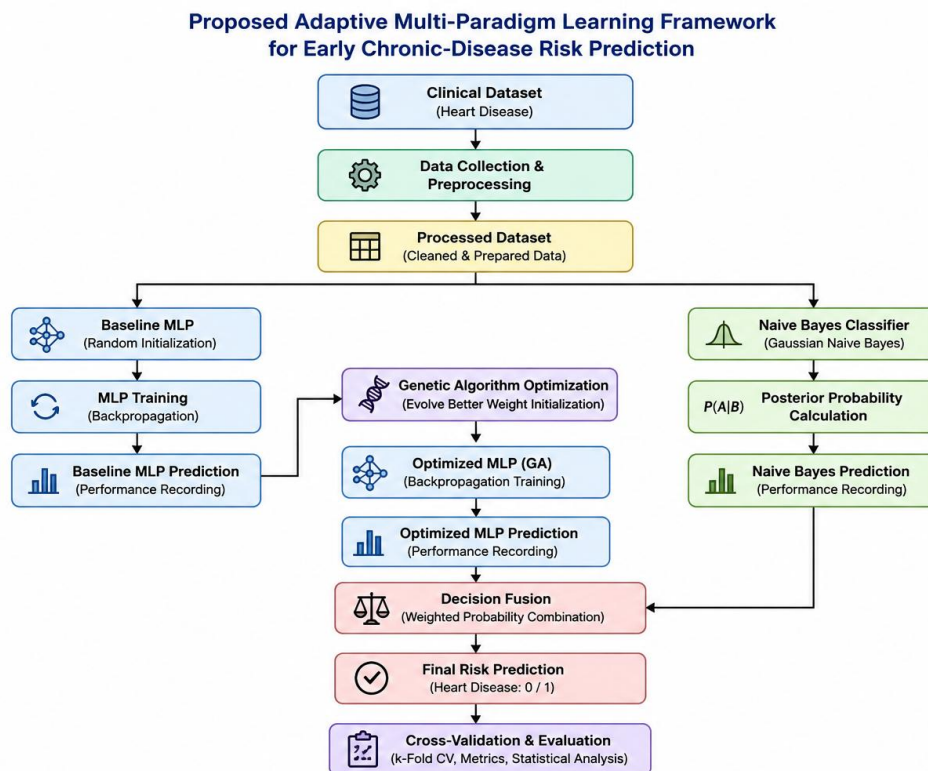


Figure 5.1: Overall Architecture of the Proposed Adaptive Multi-Paradigm Learning Framework

5.2 Baseline Multilayer Perceptron

A Multilayer Perceptron is implemented from first principles using NumPy. The core neural network implementation does not use pre-built neural network libraries such as TensorFlow, PyTorch, or Scikit-learn.

The network consists of:

- An input layer corresponding to the patient features.
- At least one hidden layer.
- An output layer for binary disease-risk prediction.

For the selected dataset, the input layer contains one neuron for each processed patient feature.

A suitable initial architecture is:

Input Layer $\rightarrow$ Hidden Layer 1 $\rightarrow$ Hidden Layer 2 $\rightarrow$ Output Layer

For example:

$$13 \rightarrow 16 \rightarrow 8 \rightarrow 1$$

The architecture may be adjusted during experimentation based on validation performance.

The baseline MLP uses randomly initialised weights and serves as the reference model for evaluating the effect of Genetic Algorithm optimisation.

5.3 Forward Propagation

During forward propagation, the input features are passed through the neural network to generate a prediction.

For each layer:

$$Z^{(l)} = A^{(l-1)}W^{(l)} + b^{(l)}$$

where:

- $A^{(l-1)}$ is the output of the previous layer.
- $W^{(l)}$ represents the weight matrix.
- $b^{(l)}$ represents the bias vector.
- $Z^{(l)}$ represents the weighted input.

An activation function is then applied:

$$A^{(l)} = f(Z^{(l)})$$

The Rectified Linear Unit (ReLU) is used for the hidden layers:

$$ReLU(z) = \max(0, z)$$

The sigmoid activation function is used in the output layer:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The sigmoid output represents the predicted probability of heart disease:

$$\hat{y} = P(Y = 1 | X)$$

A prediction is generated using a threshold:

$$Prediction = \begin{cases} 1, & \hat{y} \geq 0.5 \\ 0, & \hat{y} < 0.5 \end{cases}$$

5.4 Loss Function

Binary Cross-Entropy Loss is used to measure the difference between the actual disease label and the predicted probability.

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

where:

- y_i is the actual target value.
- $\hat{y}_i$ is the predicted probability.
- n is the number of training samples.

The objective of the learning process is to minimise this loss.

5.5 Backpropagation

Backpropagation is used to calculate how the prediction error should be distributed through the neural network.

The process follows these steps:

1. Perform forward propagation.
2. Calculate the Binary Cross-Entropy loss.
3. Calculate the gradient of the loss with respect to the output.

4. Propagate the gradients backwards through the hidden layers.
5. Calculate gradients for each weight and bias.
6. Update the parameters using gradient descent.

The parameter update rule is:

$$W = W - \eta \frac{\partial L}{\partial W}$$

$$b = b - \eta \frac{\partial L}{\partial b}$$

where η represents the learning rate.

The MLP is trained for multiple epochs until the loss decreases and the model reaches satisfactory convergence.

The convergence behaviour of this baseline model is recorded so that it can later be compared with the GA-optimised MLP.

5.6 Genetic Algorithm Optimisation

A Genetic Algorithm is implemented from scratch to improve the initial configuration of the MLP.

Instead of beginning with only one randomly generated weight configuration, the Genetic Algorithm creates a population of candidate solutions.

Each candidate solution represents a possible initial weight configuration for the neural network.

The Genetic Algorithm consists of the following components:

Population Initialisation

A population of candidate chromosomes is generated.

Each chromosome represents a flattened vector containing the MLP parameters selected for optimisation.

$$Population = \{C_1, C_2, \dots, C_n\}$$

Fitness Evaluation

Each candidate is evaluated using a fitness function.

The candidate MLP is trained for a limited number of epochs, and its validation performance is measured.

A suitable fitness function is:

$$Fitness = \alpha \times Accuracy + (1 - \alpha) \times F1$$

where α determines the relative importance of accuracy and F1-score.

A higher fitness value represents a better candidate.

Selection

Higher-performing candidate solutions are selected as parents for the next generation.

Tournament selection or rank-based selection can be used to choose parents while maintaining population diversity.

Crossover

Selected parent chromosomes exchange portions of their parameter vectors to create new offspring.

For example:

$$Child = \beta Parent_1 + (1 - \beta) Parent_2$$

where β is a crossover coefficient.

Mutation

Small random modifications are applied to selected genes.

Mutation introduces diversity into the population and reduces the possibility of premature convergence.

Generational Evolution

The process of fitness evaluation, selection, crossover, and mutation is repeated for a predefined number of generations.

The best candidate obtained from the final population is used as the optimised starting configuration for the MLP.

5.7 GA-Optimised MLP

After the Genetic Algorithm identifies an improved initial parameter configuration, the MLP is initialised using the best candidate.

The model is then trained using the same backpropagation procedure used for the baseline MLP.

This allows a controlled comparison:

Baseline Model

$$Random\ Initialisation \rightarrow MLP \rightarrow Backpropagation$$

Optimised Model

GA Optimised Initialisation → MLP → Backpropagation

The following aspects are compared:

- Initial loss.
- Rate of convergence.
- Final training loss.
- Validation performance.
- Accuracy.
- Precision.
- Recall.
- F1-score.
- ROC-AUC.

This comparison directly investigates whether GA optimisation provides a measurable improvement over conventional random initialisation.

5.8 Naive Bayes Classification

A Gaussian Naive Bayes classifier is implemented independently from first principles.

The classifier estimates the probability of each class using Bayes' theorem.

$$P(C | X) = \frac{P(X | C)P(C)}{P(X)}$$

Under the Naive Bayes assumption:

$$P(X | C) = \prod_{j=1}^d P(x_j | C)$$

where d represents the number of input features.

For continuous numerical features, the Gaussian probability density function is used:

$$P(x_j | C) = \frac{1}{\sqrt{2\pi\sigma_C^2}} \exp\left(-\frac{(x_j - \mu_C)^2}{2\sigma_C^2}\right)$$

where:

- μ_C is the mean of the feature for class C .
- σ_C^2 is the variance of the feature for class C .

The class posterior probabilities are calculated for each patient, and the class with the highest posterior probability is selected.

5.9 Manual Posterior Probability Demonstration

To demonstrate the internal operation of the Naive Bayes classifier, one patient record from the test data will be selected.

For the selected patient:

1. The class prior probabilities will be calculated.
2. The mean and variance of each feature for both classes will be obtained from the training data.
3. The likelihood of the patient's observed feature values will be calculated.
4. The posterior probability for each class will be derived.
5. The final class prediction will be determined by comparing the posterior probabilities.

This demonstration provides transparency into the probabilistic reasoning performed by the classifier.

5.10 Decision Fusion

The final stage combines the predictions of the GA-optimised MLP and Naive Bayes classifier.

Both models generate a probability representing the predicted likelihood of heart disease:

$$P_{MLP}$$

and

$$P_{NB}$$

A weighted probability fusion mechanism is used:

$$P_{final} = w_{MLP}P_{MLP} + w_{NB}P_{NB}$$

subject to:

$$w_{MLP} + w_{NB} = 1$$

The weights are determined based on validation performance.

For example, the F1-scores of the individual models can be normalised:

$$w_{MLP} = \frac{F1_{MLP}}{F1_{MLP} + F1_{NB}}$$
$$w_{NB} = \frac{F1_{NB}}{F1_{MLP} + F1_{NB}}$$

This gives greater influence to the model demonstrating stronger validation performance.

The final prediction is then determined using:

$$Prediction = \begin{cases} 1, & P_{final} \geq 0.5 \\ 0, & P_{final} < 0.5 \end{cases}$$

The fusion mechanism is evaluated to determine whether combining the two models improves predictive performance compared with the individual approaches.

5.11 Experimental Evaluation Strategy

All models are evaluated using the same preprocessing and validation framework.

The following experimental configurations are compared:

Experiment Model Configuration

Experiment 1 Baseline MLP with random initialisation

Experiment 2 MLP with GA-optimised initialisation

Experiment 3 Naive Bayes classifier

Experiment 4 Decision Fusion of MLP+GA and Naive Bayes

Stratified k-fold cross-validation is used to evaluate each configuration.

For every fold, the following metrics are calculated:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

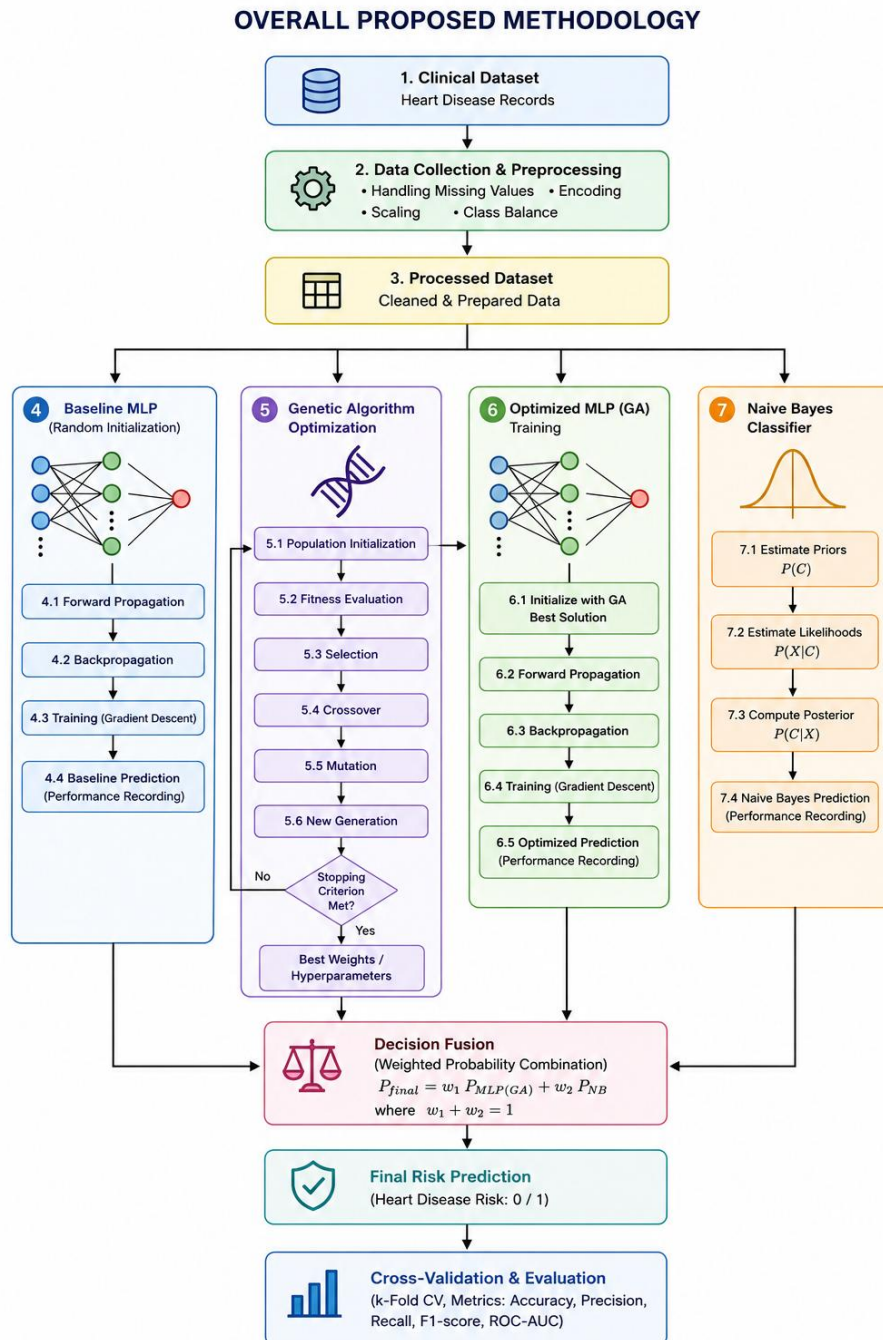
The results across all folds are aggregated to calculate:

$$\text{Mean Performance} \pm \text{Standard Deviation}$$

The fold-level results are also used for statistical comparison between the experimental configurations.

5.12 Overall Proposed Methodology

The complete methodology can be summarised as:



The proposed methodology therefore provides a structured framework for comparing neural learning, evolutionary optimisation, probabilistic reasoning, and model fusion within the same chronic-disease prediction problem.

6. MATHEMATICAL FORMULATION

6.1 Problem Representation

The chronic-disease prediction problem is formulated as a binary supervised classification problem.

Let the dataset be represented as:

$$D = \{(x_i, y_i)\}_{i=1}^n$$

where:

- $x_i \in \mathbb{R}^d$ represents the feature vector of the i^{th} patient.
- $y_i \in \{0, 1\}$ represents the corresponding disease label.
- n represents the total number of patient records.
- d represents the number of input features.

For the selected heart-disease dataset:

$$d = 13$$

The objective of the proposed framework is to learn a function:

$$f: \mathbb{R}^{13} \rightarrow \{0, 1\}$$

that predicts the probability of heart disease based on the input patient features.

6.2 Multilayer Perceptron Formulation

The Multilayer Perceptron transforms the input feature vector through multiple layers.

For the first hidden layer:

$$Z^{(1)} = XW^{(1)} + b^{(1)}$$

The activation output is:

$$A^{(1)} = \text{ReLU}(Z^{(1)})$$

where:

$$\text{ReLU}(z) = \max(0, z)$$

For the second hidden layer:

$$Z^{(2)} = A^{(1)}W^{(2)} + b^{(2)}$$
$$A^{(2)} = ReLU(Z^{(2)})$$

For the output layer:

$$Z^{(3)} = A^{(2)}W^{(3)} + b^{(3)}$$

The sigmoid activation function produces the predicted probability:

$$\hat{y} = \sigma(Z^{(3)})$$

where:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The output value satisfies:

$$0 \leq \hat{y} \leq 1$$

A binary prediction is generated using the threshold:

$$\hat{c} = \begin{cases} 1, & \hat{y} \geq 0.5 \\ 0, & \hat{y} < 0.5 \end{cases}$$

6.3 Binary Cross-Entropy Loss

The Binary Cross-Entropy loss function is used to measure the prediction error.

For n samples:

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

The objective of training is to minimise:

$$\min_{\theta} L(\theta)$$

where:

$$\theta = \{W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}, W^{(3)}, b^{(3)}\}$$

represents the complete set of trainable MLP parameters.

6.4 Backpropagation and Gradient Descent

Backpropagation calculates the gradient of the loss function with respect to each trainable parameter.

For the output layer with sigmoid activation and Binary Cross-Entropy loss:

$$dZ^{(3)} = \hat{y} - y$$

The gradients for the output-layer parameters are:

$$dW^{(3)} = \frac{1}{n} (A^{(2)})^T dZ^{(3)}$$

$$db^{(3)} = \frac{1}{n} \sum dZ^{(3)}$$

The error is then propagated to the second hidden layer:

$$dA^{(2)} = dZ^{(3)} (W^{(3)})^T$$

For the ReLU activation:

$$ReLU'(z) = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

Therefore:

$$dZ^{(2)} = dA^{(2)} \odot ReLU'(Z^{(2)})$$

The same process is repeated for the preceding hidden layer.

The parameters are updated using gradient descent:

$$W^{(l)} \leftarrow W^{(l)} - \eta dW^{(l)}$$

$$b^{(l)} \leftarrow b^{(l)} - \eta db^{(l)}$$

where η represents the learning rate.

6.5 Genetic Algorithm Formulation

The Genetic Algorithm is used to search for an improved initial configuration for the MLP.

Each candidate solution is represented as a chromosome:

$$C_i = [g_1, g_2, \dots, g_m]$$

where each gene represents an encoded parameter value associated with the MLP configuration.

The initial population is:

$$P = \{C_1, C_2, \dots, C_N\}$$

where N represents the population size.

Fitness Function

Each chromosome is evaluated using a fitness function based on predictive performance.

$$Fitness(C_i) = \alpha Accuracy_i + (1 - \alpha) F1_i$$

where:

$$0 \leq \alpha \leq 1$$

A higher fitness value indicates a better candidate solution.

Selection

Candidate solutions with higher fitness are more likely to be selected as parents.

Using tournament selection, a subset of candidate chromosomes is randomly selected, and the chromosome with the highest fitness is chosen as a parent.

Crossover

Given two parent chromosomes:

$$C_A$$

and

$$C_B$$

an offspring can be generated using arithmetic crossover:

$$C_{child} = \beta C_A + (1 - \beta) C_B$$

where:

$$0 \leq \beta \leq 1$$

Mutation

Mutation introduces a small random modification:

$$C' = C + \epsilon$$

where ϵ is a small random perturbation applied to selected genes.

The GA process is repeated over multiple generations until the stopping criterion is satisfied.

The best chromosome is defined as:

$$C^* = \arg \max_{C_i \in P} Fitness(C_i)$$

The resulting configuration is then used to initialise or configure the optimised MLP.

6.6 Naive Bayes Formulation

Naive Bayes is based on Bayes' theorem.

For class C_k and feature vector X :

$$P(C_k | X) = \frac{P(X | C_k)P(C_k)}{P(X)}$$

The Naive Bayes assumption considers the features conditionally independent given the class:

$$P(X | C_k) = \prod_{j=1}^d P(x_j | C_k)$$

Therefore:

$$P(C_k | X) \propto P(C_k) \prod_{j=1}^d P(x_j | C_k)$$

For continuous features, Gaussian probability density estimation is used:

$$P(x_j | C_k) = \frac{1}{\sqrt{2\pi\sigma_{jk}^2}} \exp\left(-\frac{(x_j - \mu_{jk})^2}{2\sigma_{jk}^2}\right)$$

where:

- μ_{jk} is the mean of feature j for class k .
- σ_{jk}^2 is the variance of feature j for class k .

The predicted class is:

$$\hat{C} = \arg \max_{C_k} P(C_k | X)$$

6.7 Decision Fusion Formulation

The proposed framework combines the probability outputs of the GA-optimised MLP and Naive Bayes classifier.

Let:

$$P_{MLP}$$

represent the probability generated by the optimised MLP, and:

$$P_{NB}$$

represent the probability generated by Naive Bayes.

The fused probability is calculated using weighted averaging:

$$P_{final} = w_{MLP}P_{MLP} + w_{NB}P_{NB}$$

subject to:

$$w_{MLP} + w_{NB} = 1$$

The weights are derived from validation performance.

Using F1-score:

$$w_{MLP} = \frac{F1_{MLP}}{F1_{MLP} + F1_{NB}}$$

$$w_{NB} = \frac{F1_{NB}}{F1_{MLP} + F1_{NB}}$$

The final prediction is:

$$\hat{y}_{final} = \begin{cases} 1, & P_{final} \geq 0.5 \\ 0, & P_{final} < 0.5 \end{cases}$$

This formulation gives greater influence to the model demonstrating stronger validation performance while retaining information from both learning paradigms.

6.8 Evaluation Metrics

The models are evaluated using the confusion matrix components:

- True Positive (TP)
- True Negative (TN)
- False Positive (FP)
- False Negative (FN)

Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

$$Precision = \frac{TP}{TP + FP}$$

Recall

$$Recall = \frac{TP}{TP + FN}$$

F1-Score

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

ROC-AUC

The Receiver Operating Characteristic curve evaluates the relationship between the True Positive Rate and False Positive Rate across different classification thresholds.

The Area Under the Curve (AUC) provides a threshold-independent measure of the model's ability to distinguish between positive and negative classes.

6.9 Statistical Comparison

The performance of the proposed models is compared across the folds generated during cross-validation.

Let the performance values of two models across k folds be:

$$M_A = \{m_{A1}, m_{A2}, \dots, m_{Ak}\}$$

and:

$$M_B = \{m_{B1}, m_{B2}, \dots, m_{Bk}\}$$

The difference for each fold is:

$$d_i = m_{Ai} - m_{Bi}$$

The mean difference is:

$$\bar{d} = \frac{1}{k} \sum_{i=1}^k d_i$$

A paired statistical test can then be used to determine whether the observed difference between the models is statistically significant.

The null hypothesis is:

$$H_0: \mu_d = 0$$

The alternative hypothesis is:

$$H_1: \mu_d \neq 0$$

A significance level of:

$$\alpha = 0.05$$

will be used. If the calculated p-value is less than 0.05, the observed performance difference will be considered statistically significant.

6.10 Summary

The mathematical formulation defines the theoretical foundation of the proposed multi-paradigm learning framework. The MLP performs non-linear classification using forward propagation and backpropagation, while the Genetic Algorithm searches for improved initial configurations. The Naive Bayes classifier provides an independent probabilistic prediction based on Bayes' theorem.

The outputs of the optimised MLP and Naive Bayes models are combined through weighted decision fusion. The resulting framework is evaluated using standard classification metrics and fold-level statistical comparison.

7. SYSTEM ARCHITECTURE AND DESIGN

7.1 System Architecture Overview

The proposed Adaptive Multi-Paradigm Learning Framework is designed as a modular Machine Learning system for chronic-disease prediction.

The system processes clinical data through multiple stages, including data preprocessing, neural-network learning, Genetic Algorithm optimisation, probabilistic classification, decision fusion, and performance evaluation.

The modular design allows each model to be trained and evaluated independently before combining their outputs.

7.2 Data Collection and Preprocessing Layer

The system begins with the collection and integration of heart-disease patient data.

The preprocessing stage prepares the dataset for Machine Learning by performing missing-value handling, data type conversion, duplicate removal, target transformation, and feature standardisation.

The processed data is then separated into input features and target labels for training and testing.

7.3 Neural Learning and Optimisation Layer

A Multilayer Perceptron is used as the primary neural-learning model.

The network receives the processed patient features and performs forward propagation through its hidden layers to generate a disease-risk probability. The prediction error is calculated using Binary Cross-Entropy loss, and the network parameters are updated through backpropagation and gradient descent.

A Genetic Algorithm is used to search for improved initial parameter configurations. The algorithm evaluates multiple candidate solutions and applies selection, crossover, and mutation over several generations.

The best configuration is used to initialise the GA-Optimised MLP.

7.4 Probabilistic Learning Layer

A Gaussian Naive Bayes classifier is implemented as an independent probabilistic prediction model.

The classifier calculates the prior probability of each class and estimates feature likelihoods using Gaussian probability distributions. Posterior probabilities are then calculated to determine the predicted disease class.

This provides a second prediction pathway that can be compared and combined with the neural-network model.

7.5 Decision Fusion and Evaluation Layer

The probability outputs of the GA-Optimised MLP and Gaussian Naive Bayes classifier are combined using weighted decision fusion.

The final prediction is generated from the combined probability output.

The system performance is evaluated using Accuracy, Precision, Recall, F1-score, and ROC-AUC. The experimental configurations include the Baseline MLP, GA-Optimised MLP, Gaussian Naive Bayes classifier, and the Decision Fusion model.

Stratified cross-validation is used to provide a consistent evaluation framework for comparing the models.

7.6 System Architecture Summary

The overall system follows the workflow:

Clinical Dataset → Data Preprocessing → Baseline MLP and Naive Bayes → Genetic Algorithm Optimisation → GA-Optimised MLP → Decision Fusion → Final Disease Prediction → Performance Evaluation

The architecture combines neural learning, evolutionary optimisation, and probabilistic reasoning within a single Machine Learning framework to investigate their individual and combined effectiveness for chronic-disease prediction.

8. IMPLEMENTATION

8.1 Implementation Environment

The proposed Adaptive Multi-Paradigm Learning Framework was implemented using Python. The implementation was organised into separate modules for data preprocessing, neural-network learning, Genetic Algorithm optimisation, probabilistic classification, decision fusion, and performance evaluation.

The following libraries were used:

Library	Purpose
NumPy	Numerical computation and matrix operations
Pandas	Dataset loading and manipulation
Matplotlib	Visualisation and performance graphs
Scikit-learn	Dataset splitting and evaluation metrics
SciPy	Statistical analysis

The major learning components, including the Multilayer Perceptron, Genetic Algorithm, and Gaussian Naive Bayes classifier, were implemented independently to demonstrate the underlying Machine Learning concepts.

8.2 Dataset Loading and Preprocessing

The heart-disease datasets obtained from the available sources were loaded and merged into a unified dataset. Consistent column names were assigned to ensure that the datasets could be combined successfully.

The preprocessing process included replacement of missing-value indicators, conversion of attributes into numerical data types, handling of missing values, removal of duplicate records, and transformation of the original diagnostic target into a binary class label.

The final processed dataset consisted of 13 input features and a binary target variable representing the presence or absence of heart disease.

8.3 Feature Standardisation and Data Preparation

Feature scaling was performed to ensure that numerical attributes with different ranges did not disproportionately influence the learning process.

A custom standardisation procedure was implemented using the mean and standard deviation calculated from the training data. The same scaling parameters were subsequently applied to the corresponding testing data.

The processed dataset was then divided into input features and target labels for model training and evaluation.

8.4 Multilayer Perceptron Implementation

A Multilayer Perceptron was implemented using NumPy as the primary neural-learning model.

The network architecture consisted of an input layer corresponding to the patient features, two hidden layers, and a single output neuron for binary classification.

The implemented architecture was:

$$13 \rightarrow 16 \rightarrow 8 \rightarrow 1$$

The hidden layers used the ReLU activation function, while the output layer used the sigmoid activation function to generate the probability of disease occurrence.

8.5 Forward Propagation and Loss Calculation

During forward propagation, the input features were passed through the hidden layers and output layer of the neural network.

The activations generated at each layer were used to calculate the final probability prediction. Binary Cross-Entropy was used as the loss function to measure the difference between the actual target value and the predicted output.

A small numerical constant was used to prevent instability caused by calculating the logarithm of zero.

8.6 Backpropagation and Model Training

Backpropagation was implemented to calculate the gradients of the loss function with respect to the network parameters.

The calculated gradients were used to update the weights and biases using gradient descent. The training process was repeated over multiple epochs, and the loss value was recorded to analyse the convergence behaviour of the model.

8.7 Genetic Algorithm Optimisation

A Genetic Algorithm was implemented to search for improved initial parameter configurations for the Multilayer Perceptron.

The algorithm generated a population of candidate solutions, evaluated their fitness, selected suitable parent solutions, and produced new candidate configurations using crossover and mutation.

The best-performing configuration was retained and used to initialise the optimised MLP.

8.8 Gaussian Naive Bayes Implementation

A Gaussian Naive Bayes classifier was implemented as an independent probabilistic learning model.

The classifier calculated the prior probability of each class along with the mean and variance of each feature. Gaussian probability estimation was then used to calculate the likelihood of each patient record belonging to the possible disease classes.

The final prediction was determined based on the calculated posterior probabilities.

8.9 Decision Fusion

The probability outputs generated by the GA-Optimised MLP and Gaussian Naive Bayes classifier were combined using weighted decision fusion.

The contribution of each model was determined based on its validation performance. The weighted probabilities were combined to generate a final disease-risk prediction.

8.10 Model Evaluation

The performance of the Baseline MLP, GA-Optimised MLP, Gaussian Naive Bayes classifier, and Decision Fusion model was evaluated using standard classification metrics.

The metrics included Accuracy, Precision, Recall, F1-score, and ROC-AUC. These measurements were used to compare the effectiveness of the different Machine Learning approaches.

8.2 Dataset Loading and Preprocessing

The heart-disease datasets from all four sources are loaded, merged and cleaned. Missing values are handled, duplicates removed, and the target variable is converted to binary class labels.

8.2.1 Loading and Merging Datasets

```
1 import pandas as pd
2 import numpy as np
3
4 columns = ["age", "sex", "cp", "trestbps", "chol",
5           "fbs", "restecg", "thalach", "exang",
6           "oldpeak", "slope", "ca", "thal", "num"]
7
8 cleveland = pd.read_csv("processed.cleveland.data", names=columns)
9 hungarian = pd.read_csv("processed.hungarian.data", names=columns)
10 switzerland = pd.read_csv("processed.switzerland.data", names=columns)
11 long_beach = pd.read_csv("processed.va.data", names=columns)
12
13 # Merge all datasets
14 df = pd.concat([cleveland, hungarian, switzerland, long_beach],
15               ignore_index=True)
16 print("Initial Dataset Shape:", df.shape)
17 df.head()
```

8.2.2 Preprocessing and Target Transformation

```
1 # Replace missing value indicators with NaN
2 df.replace('?', np.nan, inplace=True)
3
4 # Convert all columns to numeric
5 for col in columns:
6     df[col] = pd.to_numeric(df[col], errors='coerce')
7
8 # Handle missing values (median imputation for numeric columns)
9 for col in df.columns:
10     if df[col].dtype in ['float64', 'int64']:
11         df[col] = df[col].fillna(df[col].median())
12
13 # Remove duplicate records
14 initial_rows = df.shape[0]
15 df = df.drop_duplicates()
16 print(f"Duplicate Records Removed: {initial_rows - df.shape[0]}")
17
18 # Transform target column to binary class (0 = No Disease, 1 = Disease)
19 df['target'] = (df['num'] > 0).astype(int)
20 df.drop(columns=['num'], inplace=True)
21
22 print("Final Dataset Shape:", df.shape)
```

Output Summary

- | | |
|---|---|
| <ul style="list-style-type: none">Initial Dataset Shape (after merging): (Rows, 14)Duplicate Records Removed: (Number)Final Dataset Shape: (Rows, 14) | <ul style="list-style-type: none">Features: 13Target: 1 (Binary)Classes: 0 (No Disease) and 1 (Disease) |
|---|---|

Figure 8.1: Dataset Loading and Preprocessing (Implementation)

Figure 8.9: Model Evaluation Implementation

8.11 Implementation Summary

The implementation transformed the proposed methodology into a complete Machine Learning framework for chronic-disease prediction.

The process began with dataset integration and preprocessing, followed by feature standardisation and preparation. A baseline Multilayer Perceptron was implemented using forward propagation, Binary Cross-Entropy loss, backpropagation, and gradient-based learning.

The Genetic Algorithm was used to explore improved neural-network parameter configurations, while Gaussian Naive Bayes provided an independent probabilistic classification approach. The predictions generated by these models were subsequently combined through weighted decision fusion.

Finally, all experimental configurations were evaluated using common performance metrics to enable a systematic comparison of the proposed approaches.

9. EXPERIMENTAL SETUP AND EVALUATION

9.1 Experimental Setup

The proposed framework was evaluated using the processed heart-disease dataset. The experiments compared four different Machine Learning configurations:

1. Baseline Multilayer Perceptron
2. GA-Optimised Multilayer Perceptron
3. Gaussian Naive Bayes
4. Decision Fusion Model

All models were evaluated using the same processed dataset and experimental procedure.

9.2 Data Validation Strategy

Stratified 5-fold cross-validation was used to evaluate the performance of the models.

The dataset was divided into five folds while maintaining the distribution of the target classes. In each iteration, one fold was used for testing and the remaining folds were used for training.

Feature standardisation was performed using parameters calculated from the training data to prevent data leakage.

9.3 Model Configuration

The MLP used the following architecture:

$$13 \rightarrow 16 \rightarrow 8 \rightarrow 1$$

The hidden layers used ReLU activation, while the output layer used the sigmoid activation function.

The Genetic Algorithm used population-based optimisation with selection, crossover, and mutation to identify improved initial parameter configurations.

Gaussian Naive Bayes was used as the probabilistic classification model, and its predictions were combined with those of the GA-Optimised MLP using weighted decision fusion.

9.4 Performance Metrics

The models were evaluated using the following metrics:

- Accuracy
- Precision

- Recall
- F1-Score
- ROC-AUC

Confusion matrices were also used to analyse the classification behaviour of the models.

The performance values obtained from the cross-validation folds were compared to determine the effectiveness of each approach.

9.5 Experimental Summary

The experimental procedure provides a consistent framework for comparing the baseline neural model, the optimised neural model, the probabilistic classifier, and the final fusion model.

The results obtained from these experiments are presented and discussed in the following chapter.

10. RESULTS AND DISCUSSION

10.1 Experimental Results

The proposed framework was evaluated using four Machine Learning configurations:

1. Baseline MLP
2. GA-Optimised MLP
3. Gaussian Naive Bayes
4. Decision Fusion Model

The performance of the models was measured using Accuracy, Precision, Recall, F1-Score, and ROC-AUC.

10.2 Convergence Analysis

The training loss of the Baseline MLP and GA-Optimised MLP was compared to analyse the effect of Genetic Algorithm optimisation on neural-network training.

The convergence graph can be used to observe the rate and stability of loss reduction during training.

10.3 Classification Analysis

Confusion matrices were generated to analyse the classification performance of the models.

The confusion matrices provide information about correct predictions, false positives, and false negatives. This analysis is particularly useful for examining the behaviour of the models beyond overall accuracy.

10.4 Discussion

The Baseline MLP provides a reference for evaluating the effect of the optimisation process. The GA-Optimised MLP investigates whether evolutionary optimisation can improve the initial neural-network configuration and training behaviour.

Gaussian Naive Bayes provides an independent probabilistic approach to disease prediction. The Decision Fusion model combines the outputs of the GA-Optimised MLP and Gaussian Naive Bayes to investigate whether combining different learning approaches improves overall prediction performance.

The final experimental results will be used to identify the model that provides the best balance between Accuracy, Precision, Recall, F1-Score, and ROC-AUC.

10.5 Summary

The experimental results provide a comparison between the baseline neural model, the optimised neural model, the probabilistic classifier, and the proposed decision-fusion approach.

The results and visual outputs generated during implementation can be inserted into this chapter to support the final analysis and conclusions.

11. CONCLUSION AND FUTURE WORK

11.1 Conclusion

This project presented an Adaptive Multi-Paradigm Learning Framework for chronic-disease prediction using the UCI Heart Disease dataset.

The proposed framework combined multiple Machine Learning approaches, including a Multilayer Perceptron, Genetic Algorithm optimisation, Gaussian Naive Bayes classification, and weighted decision fusion.

The dataset was collected, integrated, and preprocessed before being used for model training and evaluation. A baseline MLP was developed to perform neural-network-based disease prediction. A Genetic Algorithm was then used to explore improved initial parameter configurations for the neural network.

Gaussian Naive Bayes was implemented as an independent probabilistic classification approach. The predictions generated by the GA-Optimised MLP and Naive Bayes models were subsequently combined using weighted decision fusion.

The different experimental configurations were evaluated using standard classification metrics, including Accuracy, Precision, Recall, F1-Score, and ROC-AUC.

The framework demonstrates how neural learning, evolutionary optimisation, and probabilistic reasoning can be integrated into a single Machine Learning system for analysing disease-related clinical data.

11.2 Future Work

The proposed system can be extended in several ways.

Future work may include:

- Evaluating the framework using larger and more diverse healthcare datasets.
- Extending the system to support prediction of multiple chronic diseases.
- Exploring additional optimisation techniques for neural-network training.
- Comparing the proposed approach with advanced ensemble and deep-learning models.
- Incorporating additional clinical and lifestyle-related features.
- Developing a web-based interface for interactive disease-risk prediction.
- Integrating explainable AI techniques to improve the interpretability of model predictions.

- Evaluating the framework using real-world clinical data.

These improvements could enhance the scalability, predictive performance, and practical applicability of the proposed system.

12. REFERENCES

1. D. Dua and C. Graff, “UCI Machine Learning Repository,” University of California, Irvine.
2. R. Detrano et al., “International application of a new probability algorithm for the diagnosis of coronary artery disease,” *American Journal of Cardiology*, vol. 64, no. 5, pp. 304–310, 1989.
3. S. Haykin, *Neural Networks and Learning Machines*, 3rd ed. Pearson, 2009.
4. C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
5. T. M. Mitchell, *Machine Learning*. McGraw-Hill, 1997.
6. D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley, 1989.
7. J. H. Holland, *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.
8. T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*. Springer, 2009.
9. P. Domingos and M. Pazzani, “On the optimality of the simple Bayesian classifier under zero-one loss,” *Machine Learning*, vol. 29, pp. 103–130, 1997.
10. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
11. W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61.
12. C. R. Harris et al., “Array Programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.

APPENDIX

The following figure presents selected implementation components of the proposed Adaptive Multi-Paradigm Learning Framework. The implementation includes dataset preprocessing, feature standardisation, Multilayer Perceptron training, Genetic Algorithm optimisation, Gaussian Naive Bayes classification, decision fusion, and model evaluation.

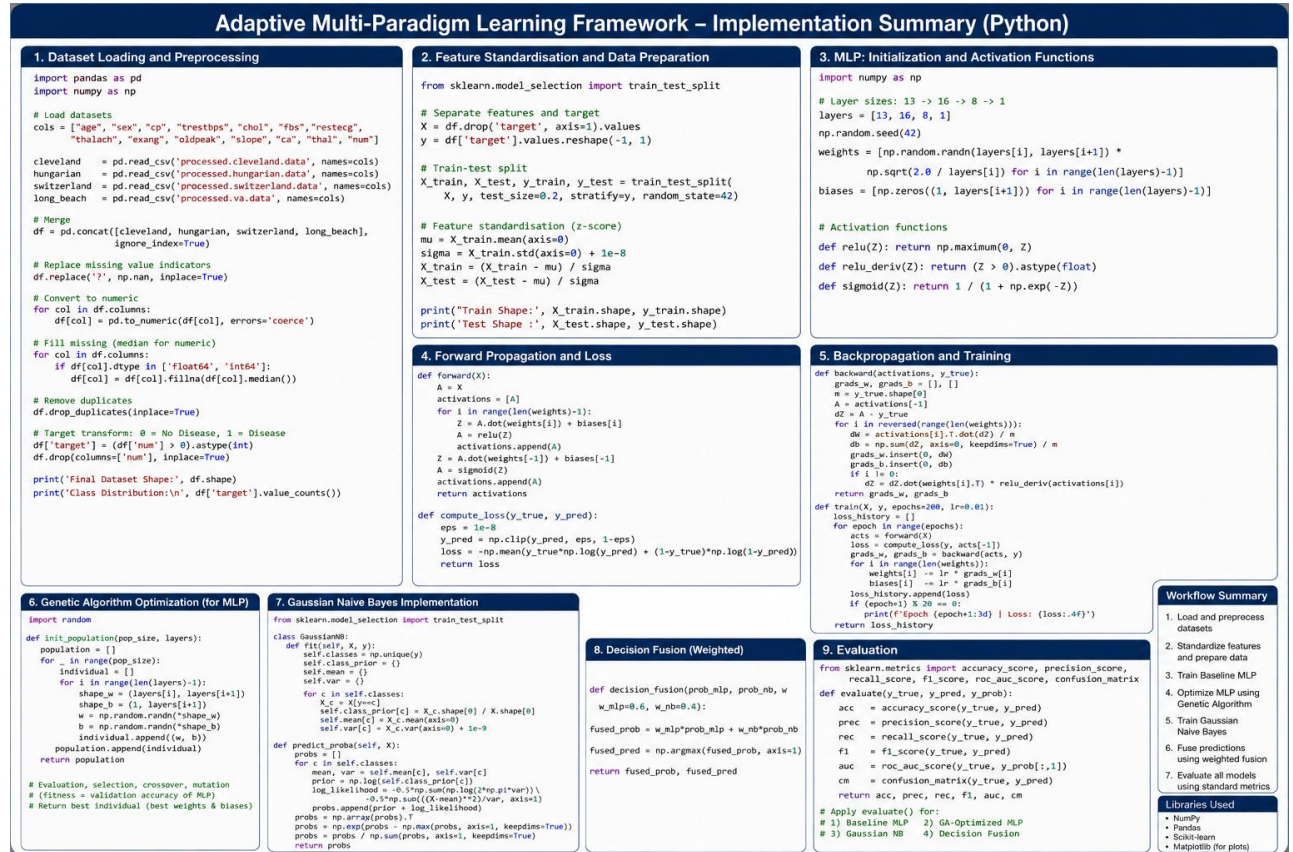


Figure A.1: Selected Implementation Components of the Proposed Framework

PROJECT SUMMARY

The project presents an Adaptive Multi-Paradigm Learning Framework for heart-disease prediction using clinical patient data.

The proposed system integrates a Multilayer Perceptron, Genetic Algorithm optimisation, Gaussian Naive Bayes classification, and weighted decision fusion. The dataset is collected and preprocessed before being used for model training and evaluation.

The framework compares a Baseline MLP, GA-Optimised MLP, Gaussian Naive Bayes classifier, and a Decision Fusion model. Their performance is evaluated using Accuracy, Precision, Recall, F1-Score, and ROC-AUC.

The project demonstrates the integration of neural learning, evolutionary optimisation, and probabilistic reasoning within a unified Machine Learning framework for disease-risk prediction.